REVISION 6 DOCKER STANDALONE

Self-Hosted Data Pipeline Site — Docker Deployment Guide

Covers Doctor v1.7: fixes the Docker Compose file-ownership guidance (Doctor now hands the env and compose files to the login user), documents what a recreate keeps and the saved rebuild recipe, adds the Source ports changes (the defined ports shown first, remove a port, editable presets, HEC on 8088 or 8001–8005, duplicate protection), corrects the service-health wording (`control-plane-init` is a one-time task), adds the host-folder volume failure to Troubleshooting, and fixes the rollback container name and two formatting defects. The behaviours new in this revision were tested on Fedora 44 with Docker 29.9.3 (2026-11-13). Rev. 5 added HEC push testing (menu options 15–16) and renumbered WEF/WEC integration to option 17.

Overview

A self-hosted Singularity Data Pipeline site can run as either a Kubernetes/K3s deployment or a **Docker standalone site** — a single container (`observo-standalone-site`) that runs every Data Pipeline service under an s6 supervisor: a one-time start-up task (`control-plane-init`) followed by five long-running services (`dataplane`, `control-agent`, `collector`, `pattern-extractor`, `monitor`). This guide covers the Docker standalone deployment model.

Networking and firewall requirements

A self-hosted site needs outbound **HTTPS (TCP 443)** to the SentinelOne cloud — no inbound connections are required for the control plane. The exact hostnames are region-specific (e.g. `na1`, `eu1`, `apac1`):

Endpoint	Purpose
`datapipeline.<region>.sentinelone.net`	Manager / auth control plane
`datapipeline-api.<region>.sentinelone.net`	API gateway
`datapipeline-logs.<region>.sentinelone.net`	Data ingestion
`datapipeline-metrics.<region>.sentinelone.net`	Metrics
Container registry (as provisioned)	Image pulls

© SentinelOne

If sources will be published on additional container ports (e.g. Syslog TCP/UDP/TLS listeners), **inbound** access to those specific host ports is also required from whatever is sending data to the site.

If your environment uses a corporate proxy, `HTTPS_PROXY` / `HTTP_PROXY` must be set on the container so it can reach the endpoints above.

Prerequisites

Before deploying a self-hosted site using Docker, ensure the following:

Infrastructure:

- A Linux host with Docker installed
- Minimum resources: 2 vCPU, 4 GB RAM, 20 GB disk
- Recommended: 4 vCPU, 8 GB RAM, 50 GB disk (supports ~1–2 TB/day throughput)

Network access:

- Outbound HTTPS (TCP 443) to SentinelOne cloud endpoints (see Networking and firewall requirements above)
- If using a corporate proxy, ability to configure `HTTPS_PROXY` and `HTTP_PROXY` environment variables

Permissions:

- Modify access to the SentinelOne scope where the site will be created
- Ability to run Docker containers on the target host
- Root or sudo access to run Docker commands

Deploy a Docker-based self-hosted site

1 Create the site in Data Pipelines

1. Sign in to your SentinelOne console.
2. Click **Data Pipelines**.
3. Go to **Pipeline Manager** > **Settings**.
4. In **Sites**, click **Add Site**.
5. Select **Self-hosted site** and click **Next**.
6. Enter a **Name** for the site (for example, "Production Syslog Collector").
7. Enter a **Description** (optional).
8. In the **Environment** dropdown, select **Docker**.

© SentinelOne

9. Select the **Scope** where this site will collect data. You can only select scopes where you have modify access.
10. Click **Next**.
11. Review the configuration and click **Done**.

The site is created and the Setup Instructions panel appears.

2 Download the configuration file

1. In the Setup Instructions panel, click **Download .env file**.
2. Save the file to your deployment host.

The `.env` file contains:

- Site ID
- Manager endpoint URL
- Authentication token
- Organization ID
- Docker registry credentials

3 Install Data Pipeline Doctor on the Linux host

Data Pipeline Doctor is the tool used both to stand up the Docker site and, afterward, to diagnose it. Install `git` and clone the repo on the Linux host that will run the container.

```
# Debian / Ubuntu
sudo apt-get update && sudo apt-get install -y git

# RHEL / CentOS Stream / Rocky / AlmaLinux
sudo dnf install -y git

# Fedora
sudo dnf install -y git

# Amazon Linux 2 / 2023
sudo yum install -y git
```

Clone the repo and make the script executable:

```
git clone https://github.com/mickbrowns1/s1-data-pipeline-doctor.git
cd s1-data-pipeline-doctor
chmod +x data-pipeline-doctor.sh
```

© SentinelOne

NOTE

If Docker isn't already installed on this host, Data Pipeline Doctor detects the OS and offers to install Docker Engine for you (Amazon Linux 2/2023, RHEL/CentOS Stream/Rocky/AlmaLinux, Fedora, Ubuntu/Debian) the first time you run it. Fedora is detected and handled as its own OS family (separate from RHEL/Rocky/Alma) since it uses a different Docker CE repo path — you don't need to do anything differently, just know it's covered.

After a fresh Docker Engine install, the installer adds your user to the `docker` group — but that only takes effect in a **new** shell session. If the wizard immediately re-checks and reports "Docker is installed but the daemon isn't reachable," don't troubleshoot Docker itself: just start a new session (re-SSH, open a new terminal, or run the tool once more with `sudo`) and re-run the command. This applies on every distro, not just RHEL.

Use Doctor's own install flow, not a generic package. If Docker is installed some other way first — for example Ubuntu/Debian's own `docker.io` package instead of letting Doctor install it — two problems show up later that look unrelated to how Docker got there: (1) the invoking user is never added to the `docker` group, so every command fails with "Docker is installed but the daemon isn't reachable" even though `dockerd` is running fine, and (2) `docker.io` doesn't ship the Compose v2 plugin, so Step 5's "Use Docker Compose?" succeeds in writing the file but then fails with `docker compose (v2 plugin) not found` when it tries to start the container. Both are confirmed live. The fix if you're already past this point: `sudo usermod -aG docker $USER` (new session required), then install Docker the way Step 3 does above — Docker's own repo includes `docker-compose-plugin` — rather than a distro package.

4 Get the .env contents onto the deployment host

The file downloaded in Step 2 is **not** named `ai-data-pipelines.env` — the console names it `Observo_<site-id>_config.env`. That mismatch, plus the fact that Data Pipeline Doctor originally only looked for that fixed filename in a specific home directory, turned out to be the single biggest source of "the wizard is stuck waiting" problems in practice: wrong home directory, wrong filename, the file landing in a `tmp` subdirectory instead, and even NFS/autofs-mounted home directories where the file was in the *right* place the whole time but a stale kernel attribute cache kept it invisible to the wizard's polling loop.

Recommended: skip the file entirely (Doctor v1.6+). When you reach Step 5, if the wizard doesn't find `ai-data-pipelines.env` already in place, it now offers to paste the `.env` contents directly into the terminal instead of waiting on a file copy. This sidesteps every issue in this section — there's no file, no path, no home directory, and no cache to get wrong. If you're going this route, all you need from this step is the `.env` contents themselves (open the downloaded file) — skip straight to Step 5 and paste when prompted. You will need to enter EOF once the file is done pasting and hit 'Enter'.

Fallback: copy the file into place. Use this if you're on an older Doctor build, or you'd simply rather have the file on disk. Data Pipeline Doctor expects the fixed filename `ai-data-pipelines.env` in the home directory of **whichever user account actually runs the script**.

© SentinelOne

If you're still using the file-copy path: `sudo` resets `$HOME` to `/root` by default (most distros ship `Defaults env_reset` / `always_set_home` in `/etc/sudoers`). Doctor v1.6+ resolves this automatically when invoked via `sudo` — it detects `$SUDO_USER` and looks in *that* user's home directory, not `/root`. It's only a **direct root login/session** (SSH'd in as `root`, or `su -` / `su root` — neither sets `$SUDO_USER`) that still falls back to `/root`. If there's any chance this host gets administered that way, copy the file into both homes to be safe.

From your local machine, copy the downloaded file to the Linux host:

```
# macOS / Linux
scp /path/to/Observo_*_config.env <user>@<linux-host>:~/downloaded.env
```

```
# Windows PowerShell — requires the OpenSSH Client optional feature
# (built into Windows 10 1809+/11; if missing:
#   Add-WindowsCapability -Online -Name OpenSSH.Client~~~~0.0.1.0)
scp C:\path\to\Observo_<site-id>_config.env <user>@<linux-host>:~/downloaded.env
```

NOTE

PowerShell doesn't expand `*` wildcards for external commands the way bash does, so `scp`'s source argument needs the exact filename you downloaded rather than a glob. If you'd rather not type it out, resolve the wildcard yourself first: `powershell scp (Get-ChildItem "C:\path\to\Observo_*_config.env").FullName ` <user>@<linux-host>:~/downloaded.env`

Then, on the Linux host, rename it into place for your login user **and** for root, and lock down the permissions on both copies (the file contains a registry auth token):

```
# Your login user's home directory
cp ~/downloaded.env ~/ai-data-pipelines.env
chmod 600 ~/ai-data-pipelines.env

# Also root's home directory, in case this host is ever administered via
# a direct root login/session rather than sudo (see the note above)
sudo cp ~/downloaded.env /root/ai-data-pipelines.env
sudo chmod 600 /root/ai-data-pipelines.env
```

NOTE

If you downloaded the file directly on the Linux host (for example, via a browser on the host itself), start from `cp ~/Downloads/Observo_*_config.env ~/downloaded.env` and run the same two copy blocks above.

5 Run Data Pipeline Doctor to deploy the site

From the cloned repo directory, run the tool and select the Docker site type:

```
./data-pipeline-doctor.sh --site-type docker
```

The tool opens with a splash screen — press **Enter** at the "Press Enter to begin..." prompt to continue.

Immediately after selecting Docker as the site type, Doctor asks once (Docker sites only) and remembers the answer in `~/.data-pipeline-doctor/profile`:

> **NOTE**
>
> **Windows Event Forwarding (WEF/WEC)** Answer yes if Windows hosts will forward their event logs to this site. Doctor then sets the site up for it (port, certificate folder, checkpoint store, FIPS) and checks it on every run.
>
> `Will this site receive Windows events over WEF/WEC? [y/N]`

Not sure? Answer no — you can add WEF/WEC to an existing site at any time from the main menu's **WEF/WEC integration** option (17) without reinstalling. This question is asked once per host; a second run on the same host skips straight past it using the saved answer.

Since no `observo-standalone-site` container exists yet, the tool then starts its **New Site Setup** wizard, which:

1. Looks for `~/ai-data-pipelines.env`. If you already copied it into place in Step 4, it's found immediately. If not, the wizard offers to let you paste the `.env` contents directly (Doctor v1.6+) — paste what you copied in Step 4, then finish with `Ctrl-D` or a line containing just `EOF`. Either way the result is `chmod 600`'d automatically.
2. Parses the embedded ECR registry, auth token, and image URI from the `.env` file and runs `docker login` against the registry automatically.
3. **If you answered yes to the WEF/WEC question**, prepares the site for it before anything else starts: - Asks for the **WEF listener port** (default `5986`) — most deployments should just accept the default; override it only if your environment specifically requires a different port (for example, a lab with more than one site sharing a host). - Creates `/etc/s1/wef` (owned by the container's user, uid 1000) and mounts it read-only into the container at the same path. - Writes and mounts the checkpoint store (`dataplane-globals.yaml`) — WEF needs one to persist bookmarks, and the dataplane has no default. - Warns that WEF cannot start with FIPS-mode crypto enabled (image 2.46.0, `PKCS12KDF`) and offers to disable it (`OPENSSL_CONF=/etc/ssl/openssl.cnf.default` in the `.env` file). This affects the **whole site**, not just WEF — don't do this in a FedRAMP/DoD or other FIPS-required environment.
4. Optionally prompts to publish additional source listener ports — pick a source type (Syslog TCP/TLS/UDP, HEC on 8088 or 8001–8005, Kafka, or a custom mapping) and then accept or change the suggested port. The same menu is available later, with a way to remove a port as well, under the main menu's **Source ports** option (see *Source ports* below). Repeating a mapping, or reusing a host port that is already taken, is refused.
5. Asks whether to manage the site with **Docker Compose** (recommended — generates `~/docker-compose-observo.yml`) or a plain `docker run`.

6. Starts the `observo-standalone-site` container using the `.env` file for configuration (`--env-file ~/ai-data-pipelines.env`), with a named `observo-data` volume for persistent state and an in-memory `/etc/secrets` (`--tmpfs`, owned by the container's user) so secrets never touch the container's disk.

NOTE

Why a named volume, and not a host folder. If you ever start the container by hand instead of through Doctor, use `-v observo-data:/var/observo/data` and `--tmpfs /etc/secrets:uid=1000,gid=1000,mode=0700`, as Doctor does. Pointing the data volume at a host folder (for example `-v /var/lib/dp-site:/var/observo/data`) fails on a fresh host: Docker creates the missing folder owned by `root`, the container runs as uid 1000, and it cannot write its checkpoint store — the container still shows `Up`, but the data plane never starts (see Troubleshooting). A named volume starts out with the folders the image ships, already owned by the right user. Reproduced on Ubuntu 24.04 and Fedora 44, with and without `sudo`.

NOTE

Certificate files: the directory matters, the filenames don't. Doctor mounts the whole `/etc/s1/wef` directory — it never assumes specific filenames. Put your server certificate, private key, and CA certificate in there under whatever names you like; the only place the exact filenames matter is the **console's own Authentication tab**, where you type the three paths explicitly. They just need to match what's actually in the directory.

If you answered yes to WEF/WEC, Doctor prints next steps once the container is up, including a **Collector address** suggestion (`<hostname>:<port>`) — this is a generic guess made before any console source exists. Once you've added the WEF source in the console, use the **real** confirmed address from **WEF/WEC integration → 3** (below) instead, which reads it back from the site's own resolved configuration rather than guessing.

If you choose Docker Compose: the generated `~/docker-compose-observo.yml` references the env file as `env_file: ~/ai-data-pipelines.env` — the `~` is expanded by whichever user's shell runs `docker compose`, not baked in as an absolute path, so manage the site (`up` / `down` / `logs` / `restart`) as the **same login user** that ran the setup wizard. The plain `docker run` path doesn't have this ambiguity — the script expands the path itself before invoking Docker.

File ownership. If you run Doctor with `sudo`, it writes both files into the login user's home (not `/root`) and hands them to that user, so `docker compose -f ~/docker-compose-observo.yml up -d` works for you without `sudo`. Earlier Doctor builds left both files owned by `root`, and the very command the wizard prints at the end then failed with `open …/ai-data-pipelines.env: permission denied`. If you set a site up with an older build, run a current Doctor once with `sudo` — it finds root-owned env/compose files, hands them back and says so — or run `sudo chown "$USER": ~/ai-data-pipelines.env ~/docker-compose-observo.yml`.

© SentinelOne

Once the container is running, re-launch the same command (`./data-pipeline-doctor.sh --site-type docker`) any time to reach the full Docker diagnostic menu — full scan, service health, TLS/DNS checks, logs, proxy/connectivity tests, and more.

Verification

Confirm the site actually came up healthy and is receiving data before handing it off.

1. **Container is running:**

   ```
   docker ps --filter name=observo-standalone-site
   ```

   Status should be `Up`, not `Restarting` or `Exited`.

2. **The data plane actually started.** `Up` only means the container's process is alive — a site can be `Up` and not running at all (for example the host-folder volume failure under Troubleshooting). Confirm the data plane deployed its configuration:

   ```
   docker logs observo-standalone-site 2>&1 | grep -c 'successfully deployed dataPlane config'
   ```

   The count should be 1 or more. `0` after a minute or two means the data plane never came up — read `docker logs observo-standalone-site`.

3. **s6 services are all up.** From the Doctor menu, option **2 — Service health (s6)** should show `dataplane`, `control-agent`, `collector`, `pattern-extractor`, and `monitor` all up. `control-plane-init` is a one-time start-up task: it runs first and then exits, so it does not appear in that list. If it hasn't succeeded yet, the other five stay down — give it a minute after first start and re-check.

4. **Run a full diagnostic scan.** Option **1 — Full diagnostic scan** runs every Docker check (service health, container/resources, TLS, DNS, volume/disk, proxy/connectivity) in one pass and will flag and offer to fix anything wrong.

5. **Site shows Online in the console.** Back in **Data Pipelines** → **Pipeline Manager** → **Settings** → **Sites**, the site created in Step 1 should show status **Online**. If it's stuck **Offline**, see Troubleshooting below.

6. **Send a test event and confirm it lands in SDL.** Option **13 — Syslog tester** sends a synthetic event (Firewall, Auth, DNS, Web, or Generic) to a published source port on `localhost`, embedding a unique UUID. Search Singularity Data Lake for `message contains '<uuid>'` to confirm end-to-end delivery. This only works if a Syslog (or CEF/LEEF) source is already configured in the console on a published container port — add one from **Source ports**, option 12, if you haven't yet; Doctor reads the dataplane config to confirm one actually exists and refuses with a clear message if it doesn't, rather than offering a port that has no real source behind it.

7. **HEC push works the same way.** Option **15 — HEC tester** sends a real `POST /services/collector/event` to a configured Splunk HEC-push source, reading its address, TLS setting, and token straight from the

dataplane config — same gating as the Syslog tester (no source configured means it refuses, rather than guessing a port). Option **16 — HEC stream (continuous)** sends the same event on a loop. Both label each test event with which pipeline it went to (`pipeline=<entity-id>:<port>`), so a site with more than one Syslog or HEC source stays distinguishable in SDL.

Example, run against a live site with two HEC sources:

```
HEC Tester — observo-standalone-site

Which HEC source to send to?
  1) host 8001 → container 8001  (HEC push)
  2) host 8002 → container 8002  (HEC push)
  3) Back

Choose [1-3]: 1

[INFO]  Sending to http://localhost:8001/services/collector/event (pipeline:
10000000000000004413:8001)

[ OK ]  HEC accepted the event (HTTP 200).
Watch it arrive: Docker menu option 10 — Tail sources & destinations.

Use the following in SDL Event ALL Search to locate the test event: message contains
'240ea7d0-4816-49f2-9abc-0372de6b199a'
```

Source ports (menu option 12)

Docker can't add or remove a published port on a running container, so both actions **recreate** it: a Compose site has its `~/docker-compose-observo.yml` edited and `docker compose up -d` run, a plain `docker run` site is rebuilt with the new port list. Either way the old container or file is kept as a backup first and a rollback is printed, and nothing happens until you confirm. Option 12 opens by listing the ports the container already publishes, with each one's role, and then asks **Add a port**, **Remove a port** or **Back**.

Add a port

Pick the source type. Each suggests a port, but the suggestion is only a default — at the prompt press Enter to keep it, or type a different port.

Source type	Suggested port
Syslog TCP	`1514`
Syslog TLS	`6514`

Source type	Suggested port
Syslog UDP	`514/udp`
HEC push	`8088`, or `8001` – `8005` (you choose which)
Kafka	`9092`
Custom	you enter the protocol and port

The same list and prompt appear in New Site Setup's port step, so a site that wants HEC on `8003` rather than `8088` doesn't need "Custom".

The prompt accepts `8003` (host and container port the same, which is what you want), `5514:514` (`host:container`), or a `/udp` suffix such as `514/udp`. **The container port must match the listen address of the source you configure in the console** — the dataplane binds the literal address, so publishing `5514:10514` for a source listening on `514` sends traffic to a port nothing is listening on (UDP gives no error). Ports must be 1–65535.

Doctor refuses a mapping that is already published, or that reuses a host port and protocol already in use, instead of writing it twice. It doesn't matter whether the port or the console source is created first; data flows once both exist.

Remove a port

Remove a port lists what is published, with each port's role:

```
1) host 514 → container 514/udp    Syslog UDP
2) host 6001 → container 6001/tcp
3) host 8003 → container 8003/tcp
4) Back
```

Pick one and Doctor first checks whether a source is still listening on it in the dataplane config. If one is, it warns and asks before going on (the default is no):

```
[INFO]  Removing: host 8003 → container 8003/tcp
  This only stops publishing the port on this host. It does not remove or change the source in
the console.
[WARN]  A source is still listening on container port 8003/tcp — once the port is unpublished,
nothing can send to it and its data stops.
Remove it anyway? [y/N]
```

Removing a port **only unpublishes it on this host**. It does not delete or change the source in the console — remove or repoint that there if you no longer want it. On a Compose site, every entry for that host port is removed

from the file (including an accidental duplicate), and the `ports:` key is dropped if the list ends up empty. If the port isn't found in the compose file — for example it was published some other way — Doctor says so and changes nothing.

WEF/WEC integration (menu option 17)

If this site receives Windows Event Forwarding/Windows Event Collector traffic, option **17 — WEF/WEC integration** has five tools. Everything below is real output from a live, successful run on a fresh site — not illustrative text.

For the full certificate-generation and Windows-host configuration walkthrough (OpenSSL CA/server/client certs, WinRM, the Subscription Manager registry value, Event Log Readers), see the companion **Windows Event Forwarding (WEF) to Data Pipeline Site Setup Guide**. This section covers what Doctor itself does.

1. Readiness check (read-only)

Reads the site's own resolved configuration and reports OK/WARN/FAIL — nothing here changes anything. A healthy site looks like this:

```
WEF/WEC Readiness Check — observo-standalone-site
Read-only — nothing here changes anything.

[ OK ]  WEF source found — listen 0.0.0.0:5990, callback wef.lab.test:5990
[ OK ]  Container port 5990 is published on the host
[ OK ]  Checkpoint store config is mounted
[ OK ]  Readable inside the container: /etc/s1/wef/ca.crt
[ OK ]  Readable inside the container: /etc/s1/wef/server.crt
[ OK ]  Readable inside the container: /etc/s1/wef/server.key
[ OK ]  FIPS mode disabled for WEF (OPENSSL_CONF=/etc/ssl/openssl.cnf.default)
[ OK ]  No WEF error patterns in recent logs
[ OK ]  Forwarders seen recently: DESKTOP-JB76LQO

No blocking issues found (0 warning(s)).
```

If any line shows `[FAIL]`, run option 2 below — it's built to fix exactly these checks.

2. Make this site WEF-ready (one recreate)

Diagnoses what's missing against the same checks as option 1, then fixes everything it can in a single container recreate (Compose-aware, timestamped backup, existing rollback safety net): checkpoint store, certificate mount, and FIPS. On an already-healthy site it's a no-op:

```
Make this site WEF-ready — observo-standalone-site

[ OK ]  Already WEF-ready — nothing to do.
```

3. Windows forwarder setup

Prints the exact values for configuring a Windows forwarder, read back from the site's own confirmed configuration (not a guess), and offers to write them as a ready-to-run PowerShell script:

```
Windows forwarder setup — observo-standalone-site
What to configure on each Windows host that forwards events to this site.

Values for this site
  Collector address:   wef.lab.test:5990
  CA thumbprint:       1FD6C48D95D1E7C449FA7EA7A1E481F4D31799A0
  Subscription manager:
    Server=HTTPS://wef.lab.test:5990/wsman/SubscriptionManager/
WEC,Refresh=60,IssuerCA=1FD6C48D95D1E7C449FA7EA7A1E481F4D31799A0

On each Windows forwarder (elevated PowerShell)
  1. wef.lab.test must resolve to this host (DNS, or a hosts-file entry for a lab).
  2. Import the CA into Local Computer → Trusted Root Certification Authorities.
  3. Import a client certificate (.pfx, Client Authentication EKU, issued by that CA)
     into Local Computer → Personal, and grant NETWORK SERVICE read on its private key.
  4. Add NETWORK SERVICE to the 'Event Log Readers' group (needed for the Security log).
  5. Set WinRM to Automatic and start it.
  6. Set the subscription manager to the value above (GPO or, without a domain, the registry
value it writes).
  7. Restart WinRM, then check Event Viewer → ... → Forwarding → Operational: 104 = connected,
105 = error.

Write these steps as a PowerShell script to ~/wef-forwarder-setup.ps1? [y/N] y
[ OK ]  Wrote ~/wef-forwarder-setup.ps1
Copy it to a forwarder and run in an elevated PowerShell:
  powershell -ExecutionPolicy Bypass -File .\wef-forwarder-setup.ps1 -PfxPath .\client.pfx
-CaPath .\ca.crt
```

4. TLS endpoint test

Checks the certificate chain and does a real TLS handshake against the container port:

```
WEF TLS endpoint test — observo-standalone-site
Checks the certificates the WEF source uses and does a real TLS handshake.

[INFO]  Server certificate: CN = wef.lab.test  (SAN: wef.lab.test; expires Jan 31 14:03:34 2029
GMT)
[ OK ]  Certificate covers the callback host wef.lab.test
[ OK ]  Certificate is signed by the configured CA
[ OK ]  Private key matches the certificate
[ OK ]  Certificate has the Server Authentication EKU
[INFO]  CA thumbprint (IssuerCA= on forwarders): 1FD6C48D95D1E7C449FA7EA7A1E481F4D31799A0

[ OK ]  TLS handshake on container port 5990 succeeded (TLSv1.2, verified against the CA)
[ OK ]  Host port 5990 accepts connections
[WARN]  wef.lab.test doesn't resolve from this host — fine if only the forwarders' DNS or hosts
files have it

[ OK ]  TLS endpoint looks good. If forwarders still don't connect, check their side (option
3).
This test's own handshake appears once in the site logs as a TLS handshake failure; that's
expected.
```

5. Forwarder activity

Summarizes connections, event volume, and TLS failures per forwarder over a chosen lookback window (default 24 hours):

```
WEF forwarder activity — observo-standalone-site

Forwarders (by client certificate name)
DESKTOP-JB76LQO                 4 connection(s), last 18:11:08Z

Events received  (per hour, UTC)
13 event(s) in 3 batch(es), last batch 18:09:29Z
  18:00  13

TLS handshake failures
    6  unexpected EOF (Windows probe connection — normal)
    1  no client certificate sent (Doctor TLS test, or a forwarder missing its client cert)
    3  timed out
    1  session resumption refused (Windows retries with a full handshake — normal)

Recent WEF errors
(none)
```

NOTE

Forwarders are named by the CN of their client certificate — hosts sharing one certificate appear as a single entry. Remote addresses are the Docker bridge (port publishing NATs them), so they don't identify a forwarder. Most TLS handshake "failures" above are benign — Windows probe connections and session-resumption retries are routine on a healthy site; only a sustained pattern of real failures (bad cert, wrong CA) needs attention.

Troubleshooting

Start with option **1 — Full diagnostic scan** — it runs every check below in one pass and offers fixes automatically where possible. The rest of this section maps specific symptoms to the right menu option and, where relevant, notes what the tool does about it.

Symptom	Likely cause	Doctor menu option
Site shows **Offline** in the console	Container not running, or can't reach the Manager	2 Service health · 3 Container & resources · 11 Proxy & connectivity
control-plane-init failing / all 5 services down	Connectivity or TLS to the control plane	1 Full scan · 11 Proxy & connectivity (auto-classifies connectivity vs. TLS vs. auth, then offers a fix)
Data not flowing to AI SIEM	Logs-ingestion egress blocked (firewall/ proxy)	11 Proxy & connectivity (tests the `-logs` / `-metrics` / `-api` endpoints specifically, not just auth)
TLS handshake failures	CA not trusted / cert mismatch	4 TLS & certificates — fixes by showing the required `-v <ca-bundle>:...:ro` + `SSL_CERT_FILE` recreate change
Connection refused on a source port	Port not published, or host firewall blocking it	12 Source ports — shows current mappings, adds or removes them, prints firewall hints

Symptom	Likely cause	Doctor menu option
Syslog tester reports success, but the event never shows up in SDL	A site set up before 2026-11-02 may have a Syslog port published to the *wrong* container port — the New Site Setup wizard's old presets mapped `514:10514`, assuming the dataplane remaps a privileged port internally. It doesn't: the dataplane binds directly to the literal address the console source is configured with. UDP gives no error when nothing is listening on the other end, so the tester reports success regardless	12 Source ports to check the mapping; compare against `cat /etc/dataplane/config.yaml`'s `address:` for that source — host and container port should match exactly. Fix by re-publishing with host:container equal (e.g. `514:514/udp`) and recreating. Current Doctor builds publish new Syslog ports correctly by default
Events dropped	Disk queue full / memory exhausted	6 Volume & disk · 3 Container & resources
Container OOMKilled	Memory limit too low for throughput	3 Container & resources — bumps the limit live via `docker update --memory` (or prints a recreate command)
High CPU	Pipeline too complex, or throughput over sizing	3 Container & resources (limits vs. live `docker stats`)
Setup wizard stuck at "Waiting for ~/ai-data-pipelines.env..." despite the file being there	Usually one of: wrong home directory (direct root login vs. `sudo` — Step 4), wrong filename (still named `Observo_<site-id>_config.env`), or the file is on an NFS/autofs-mounted home directory and a stale kernel attribute cache is hiding it from the polling loop	Ctrl-C out and re-run with the **paste** option instead (Doctor v1.6+) — it doesn't poll for a file at all, so none of these causes apply. Otherwise: `find ~ /tmp -iname 'ai-data-pipelines.env'` to locate it, and `mount | grep -i home` to check for NFS/autofs
`docker compose up` **fails with "env file ... not found"**	Compose was run by a different user/ `$HOME` than expected, and no copy of the `.env` file exists there	Copy `ai-data-pipelines.env` into that user's home directory too, or always invoke Compose as the same user that ran setup (see the callout in Step 5)
`docker compose` **fails with "open …/ai-data-pipelines.env: permission denied"**	The env and compose files were created by an older Doctor run under `sudo` and are owned by `root`, so your login user can't read the env file	Run a current Doctor once with `sudo` (it hands root-owned env/compose files back to you and says so), or `sudo chown "$USER": ~/ai-data-pipelines.env ~/docker-compose-observo.yml`

Symptom	Likely cause	Doctor menu option
Container shows `Up` but the site never goes Online; logs show `panic: checkpoint store: create checkpoint data dir … permission denied` (and `data_dir "/var/observo/data/monitor" does not exist`)	The container was started by hand with a **host folder** for the data volume (for example `-v /var/lib/dp-site:/var/observo/data`). Docker creates the missing folder owned by `root`, but the container runs as uid 1000 and can't write to it, so the data plane never starts while `docker ps` still says `Up`	Use the named volume Doctor uses — `-v observo-data:/var/observo/data` — or let Doctor create the site (Step 5). Verify with the check in Verification step 2 (`successfully deployed dataPlane config` count of 1 or more)
A single s6 service is down but the container is otherwise healthy	One service crashed independently	8 Service debugger → per-service logs, then 9 Restart service / container (restart just that service, no need to recreate the container)
WEF source fails: "No checkpoint store configured"	The dataplane has no default checkpoint store — a known product gap, not a misconfiguration	17 WEF/WEC integration → 1 Readiness check to confirm, then 2 Make this site WEF-ready
WEF source fails: "Could not build PKCS#12 archive ... PKCS12KDF"	FIPS-mode crypto is enabled; WEF cannot build its TLS identity on image 2.46.0 with FIPS on	17 WEF/WEC integration → 2 Make this site WEF-ready (disables FIPS site-wide, with its own warning and confirmation — not for FedRAMP/DoD)
Forwarder logs `id=105` ("having a problem communicating with subscription manager"), but Readiness check (17 → 1) and TLS endpoint test (17 → 4) both pass	The site itself is fine — the forwarder can't actually reach the collector address over the network (DNS/hosts resolution, routing, or a firewall on the path). This is easy to miss since every site-side check is green	From the **forwarder itself**, not the site: `Test-NetConnection <collector-address> -Port <port>` in PowerShell. `TcpTestSucceeded: False` confirms it's a network path problem, not a site or cert problem — fix DNS/hosts/routing on the forwarder's side
No HEC source found when running the HEC tester	No Splunk HEC-push source is configured in the console yet (or its container port isn't published)	15 HEC tester explains which is missing; add the source in the console, or 12 Source ports to publish it
HEC tester reports HTTP 401/403	The token Doctor read from the dataplane config doesn't match what the source expects — usually the token was rotated in the console after the site's config was last reloaded	Re-check the token in the console's source config; restart the site if it was just changed

Docker-specific auto-fixes the tool applies without a recreate:

Issue	Fix
`control-plane-init` retry loop	Classifies connectivity / TLS / auth, routes to the matching remedy, then offers a restart
Connectivity failure (`EOF` / refused / timeout to the auth endpoint)	Shows the required egress and the exact `docker run -e HTTPS_PROXY=...` change, then offers a restart
TLS CA trust (`certificate signed by unknown authority`)	Shows the required `-v <ca-bundle>:...:ro` + `SSL_CERT_FILE` recreate change, then offers a restart
OOMKilled	Bumps the memory limit live with `docker update --memory` (falls back to a recreate command if that fails)
A single s6 service down	Restarts just that service (`s6-svc`) or the whole container

NOTE

A running container's mounts and environment can't be changed in place — TLS and proxy fixes print the exact recreate command and offer to restart the container for you; they don't silently recreate it behind your back.

If a fix requires recreating the container (new mount, new env var, new published port), the tool always renames the old container as a backup first, so a bad recreate is reversible. The backup is named `observo-standalone-site_pre<timestamp>` and the exact name is printed when it is made (the "show me the command" option prints `_old` instead). To roll back:

```
docker rm -f observo-standalone-site
docker rename observo-standalone-site_pre<timestamp> observo-standalone-site
docker start observo-standalone-site
```

What a recreate keeps

Docker can't change most settings on a running container, so a recreate rebuilds the `docker run` from the live container. Doctor carries over the image, environment, volumes, published ports, restart policy, memory and CPU limits and the in-memory `/etc/secrets`, **and** the settings a hand-built site is likely to have: network mode and aliases, DNS and `--add-host` entries, user and groups, ulimits, capabilities, `--privileged`, `--security-opt`, `--read-only`, sysctls, devices, log options, pids/shm/CPU limits, an explicit hostname, working directory, stop signal and timeout, labels, and an entrypoint or command override. Each is restated only when it differs from the image's own default, so an unmodified site's command is unchanged.

© SentinelOne

A few things a plain flag can't restore — a second network, a custom health check, IPC/PID/UTS/userns modes, `--runtime`, `--volumes-from`, `--link`, `-P`, an inline seccomp profile. Doctor lists these in a warning **before** the recreate instead of dropping them silently; add them back by hand afterwards.

If the container (or the host) is lost

A recreate reads the live container, so it can't help once the container is gone. For a plain `docker run` site, Doctor therefore keeps a saved recipe beside the env file it already writes:

File	Contents
`~/.data-pipeline-doctor/<container>.recipe`	The full `docker run` command. **No secrets inline.** Anything it couldn't capture is listed as `# NOT CAPTURED`.
`~/.data-pipeline-doctor/<container>.env`	The site's environment, including the auth token (mode `600`, folder mode `700`).

The recipe is retaken whenever the container changes (the previous five versions are kept as `<container>.recipe.<timestamp>`) and its location is shown in the menu header. If Doctor starts and finds **no running container** but a saved recipe, it offers to rebuild the site exactly as it was, after checking that the env file exists, that the image is present (pulling it if not — this needs a valid registry login, and the token in the `.env` file expires), and that host paths the container mounts exist (a missing one would be created empty and owned by `root`, which breaks the site).

NOTE

Docker Compose sites don't need a recipe: `~/docker-compose-observo.yml` is the recipe, and `docker compose up -d` rebuilds the site from it. Keep that file and the env file it points at. Because the env file holds the site's secrets, treat the recipe folder like the `.env` you downloaded — and note that Doctor now writes the env copy the first time it sees a plain `docker run` site, not only when it recreates one.

© SentinelOne